

Hylograph Visualization Keys Library

Status: Design / requirements **Date:** 2026-03-29 **Scope:** New package in purescript-hylograph-libs

Problem

Every Minard visualization that uses color, shape, or size encoding needs a key (legend) to explain what the visual channels mean. Currently these are hand-coded SVG in each component, leading to:

- 1. **Inconsistency:** Declaration kind legend (value/data/newtype/class/synonym/foreign) is static — doesn't update when Reachability, Purity, or Git overlays change the meaning of colors
- 2. **Duplication:** At least 4 separate legend implementations (SceneCoordinator footer, AnatomyBeeswarm inline SVG, ModuleBeeswarm, ModuleAnatomyViz)
- 3. **Wrong placement:** Declaration legend appears on deprecated pages (DeclarationDetail) and pages where it's only partially relevant
- 4. **No interactivity:** Legends are passive labels — no hover-to-highlight, no filtering
- 5. **No composition:** When overlays compose (e.g. Uses + Reachability), there's no way to show what both encodings mean simultaneously

Current Inventory (Minard)

Location	What	Interactive	Overlay-aware
SceneCoordinator footer	Declaration kinds (6 colored dots)	No	No
AnatomyBeeswarm	Package categories (Your code/Direct/Transitive/Unused)	No	No
ModuleBeeswarm	Module categories	No	No
ModuleAnatomyViz	Custom anatomy legend	No	No
PackageAnatomyViz	Decomposition legend	No	No
ModulePlanetViz declaration dots	None — tooltip only	N/A	N/A

Design Goals

1. Declarative specification

A key is data, not hand-coded SVG:

```
type KeyEntry =
  { color :: String
  , label :: String
  , shape :: Shape      -- Circle | Square | Diamond | Line | Dashed
  , group :: Maybe String -- For CoordinatedHighlight integration
```

```

}

type KeyConfig =
  { entries :: Array KeyEntry
  , layout :: KeyLayout
  , interactive :: Boolean -- Enable hover-to-highlight
  , title :: Maybe String
  }

data KeyLayout
  = HorizontalStrip -- Inline row of items (current footer style)
  | VerticalBox -- Stacked column (current anatomy style)
  | Grid Int Int -- Bivariate matrix (rows × cols)

```

2. Overlay-aware / reactive

The key should be a function of the current visual state, not a static element. When the overlay changes, the key updates:

```

Default mode:      ● value ● data ● newtype ● class ● synonym ●
foreign
Reachability (R): ● reachable ● unreachable ● entry point
Purity (P):       ● pure ● effectful
Git (G):          ● modified ● staged ● untracked ● clean
Coupling (C):     ● low coupling ... ● high coupling (gradient)

```

This means the key entries are computed from the current **ColorMode** / overlay state, not hardcoded per view.

3. Interactive (legend-as-filter)

Hovering a key entry highlights all matching marks in the visualization. This uses the existing CoordinatedHighlight system — each key entry's **group** field maps to the highlight group.

Inspiration: [bivariate choropleth](#) where hovering a cell in the 3×3 legend grid highlights all matching map regions.

For Minard, this would mean: hover "unreachable" in the Reachability key → all unreachable modules dim/highlight. Already partially possible via CoordinatedHighlight groups, just needs wiring.

4. Layout variants

- **HorizontalStrip**: Current footer style. Good for 3-8 items. Wraps on small screens.
- **VerticalBox**: Current anatomy style. Good for sidebar or floating panel.
- **Grid**: For bivariate/multivariate encodings. N×M matrix of color swatches. Each cell hoverable independently.

All layouts should be configurable for:

- Position: footer, sidebar, floating (absolute), inline (within the viz SVG)
- Size: compact (9px labels), standard (11px), large (13px)
- Background: transparent, white card, themed

5. Composable

Multiple keys can coexist when multiple encodings are active:

```
[Declaration kinds: ● value ● data ● newtype ● class]
[Overlay: ● reachable ● unreachable ● entry point]
```

Or for bivariate, a single Grid key showing the intersection.

6. HATS integration

The key should be renderable as either:

- A HATS **Tree** (for embedding inside SVG visualizations)
- A Halogen **HTML** element (for embedding in component HTML around the viz)

Both should support CoordinatedHighlight behaviors.

Implementation Plan

Phase 1: Core types and pure rendering

- Define **KeyEntry**, **KeyConfig**, **KeyLayout** types
- Pure rendering functions: **renderKeyHATS :: KeyConfig -> Tree** and **renderKeyHTML :: KeyConfig -> HTML**
- No interactivity yet — just correct, styled, declarative keys
- Test with Minard's declaration kind legend as first migration

Phase 2: CoordinatedHighlight integration

- Add **group** field to KeyEntry
- Hovering a key entry triggers highlight in the associated group
- Dimming/brightening follows existing HATS highlight semantics

Phase 3: Overlay-aware key computation

- Define **computeKey :: ColorMode -> OverlayState -> KeyConfig** in Minard
- Wire into SceneCoordinator so the key updates reactively
- Remove all hand-coded legends from individual components

Phase 4: Bivariate grid keys

- Grid layout with N×M cells
- Each cell independently hoverable
- Useful for composed overlays (e.g. Purity × Reachability)

Multivariate Encoding Reference

Per Tamara Munzner (Visualization Analysis & Design), the channels available for encoding multiple variables simultaneously on marks:

Channel	Best for	Separability
Color hue	Categorical (≤ 8 values)	High
Color luminance	Ordered/quantitative	High
Size/area	Quantitative	High
Shape	Categorical (≤ 6)	High with color
Stroke weight	Ordered	Medium
Stroke style (solid/dashed)	Binary/categorical	Medium
Opacity	Ordered	Low (interferes with color)
Position	Quantitative	Highest

Key principle: Use channels that are perceptually separable. Color hue + size is easy to read simultaneously. Color hue + color luminance is hard (they interfere). This is why bivariate color ramps are notoriously difficult to interpret — the legend-as-filter approach compensates by making the legend the primary reading tool rather than trying to decode the color directly.

For Minard: Current overlays use color hue exclusively (one at a time). Composable overlays should use separable channels — e.g. color for one dimension, stroke weight or shape for another. The Uses (U) overlay already does this well: orange curved lines are a completely different visual channel from module cell fill color.

Additional Design Ideas

Key as controller, not just label

- **Click to isolate:** click a key entry to show only matching marks, hide everything else. Click again to restore. Turns the legend into a filter bar.
- **Shift-click to accumulate:** add categories to the visible set (standard Tableau/Observable pattern).

Contextual counts

Each entry shows how many marks match: ● reachable (45) ● unreachable (12) ◆ entry point (1). Immediately answers "how many?" without scanning the viz. Could render as a number or a proportional bar behind the entry.

Continuous/gradient keys

For quantitative encodings (blame age, coupling score, change frequency), support gradient strips with labeled breakpoints: [old  recent] with tick marks. Not just discrete swatches.

Animated transitions

When switching overlays, the key should morph — entries cross-fade, reorder, change color — not jump-cut. Reinforces that the same marks are being re-encoded, not replaced.

Accessibility

Pattern fills or small symbols alongside colors for colorblind users. The declaration kinds already use shapes somewhat (circles vs dots) but not systematically. The key library should support `shape` as a first-class channel, not just `color + label`.

Coordinated keys across views

When the same dataset appears in multiple visualizations (treemap and beeswarm showing the same packages), they should share a key. Hovering in the key highlights in both views simultaneously. Falls out naturally from CoordinatedHighlight groups if group names are consistent across views.

Key generation from data

For continuous or high-cardinality data, the library could compute appropriate bins from the actual distribution — e.g. "there are 3 natural clusters in coupling scores, use those as breakpoints" rather than arbitrary quartiles.

Nested/hierarchical entries

Key entries that expand into sub-categories: ► `Git` → ● `modified (3)` ● `staged (1)` ● `untracked (2)` ○ `clean (75)`. Collapsible sub-entries within a category.

Responsive behavior

At narrow widths, collapse the key to an icon that expands on click/hover. At wide widths, show the full strip. The key shouldn't compete with the visualization for space.

Two-Tier Architecture

Following the same pattern as `hylograph-simulation` / `hylograph-simulation-halogen`:

Tier 1: `hylograph-key` (pure / HATS)

- Pure functions: `KeyConfig` → `Tree`
- No state, no effects, no subscriptions
- Embeds directly inside any SVG visualization
- Good for: static keys, export/screenshot, non-Halogen consumers, server-side rendering
- Handles: layout, styling, shape rendering, gradient strips
- CoordinatedHighlight behaviors attached declaratively (same as how treemap cells get highlight behaviors)
- **Zero Halogen dependency**

Tier 2: `hylograph-key-halogen` (component wrapper)

- Wraps Tier 1 with Halogen component lifecycle

- Manages: active/filtered entries, click-to-isolate state, expand/collapse, animated transitions between overlay modes
- Subscribes to overlay state changes from parent via `Input`
- Emits outputs: `EntryHovered String`, `EntryClicked String`, `FilterChanged (Set String)`
- Handles: responsive collapse, nested/hierarchical expand, count updates
- Depends on `hylograph-key` + `halogen`

Why two tiers

- Tier 1 is useful *inside* HATS visualizations (e.g. the anatomy beeswarm legend is SVG-internal)
- Tier 2 is useful *around* them (e.g. the SceneCoordinator footer legend is HTML)
- Tier 1 is testable without a DOM
- Clean dependency direction: `hylograph-key` has no framework dependency

Open Questions

1. How to handle very large categorical sets (e.g. co-change clusters with 8+ colors)? Truncate? Scroll? Paginate?
2. Should key generation from data (automatic binning) live in the key library or be a separate concern?
3. Relationship to Hylograph's existing tooltip system — should key hover use tooltips or just highlighting?
4. Should Tier 1 support CSS-in-HATS theming, or take explicit color/font config?
5. How to handle keys for encodings that are partially spatial (e.g. "left = more dependencies, right = fewer") — positional legends?